Optimization and Machine Learning in VLSI Design Automation
Peking University, Fall 2026
Lecturer: Yibo Lin
Teaching Assistants: Shenglu Hua, Haichuan Liu
Release Date: October 12, 2026, 20:30
Due Date: October 27, 2026, 0:00

Homework 2: STA

Notation

Arrival time is AT, required arrival time is RAT, and $slack = RAT - AT$. All times in Problems 1–4 use the units shown in the figures; all times in Problem 5 use ns.

Problem 1: Elmore delay

The input is a rooted RC tree. Each node v has intrinsic capacitance C_v and each directed edge (u, v) has resistance R_{uv}. Define

$$C_{\downarrow}(v) = C_v + \sum_{(v,w)\in E} C_{\downarrow}(w), \qquad D(r) = 0, \qquad D(v) = D(u) + R_{uv}C_{\downarrow}(v).$$

Write pseudocode that computes $C_{\downarrow}$ by post-order traversal and D for every node by pre-order traversal. The input is a rooted acyclic tree with nonnegative resistances and capacitances.

```
struct RctEdge {
  RctNode *fr, *to;
  float res;
};

struct RctNode {
  // input
  float ncap;
  std::list<RctEdge *> fanout;
}
```

Figure 1: RC-tree data structure

Problem 2: Arrival time and slack

Primary inputs A, B, C, D arrive at time zero. The gate delays and connectivity are given in Figure 2; the primary output is I and $RAT(I) = 50\,\text{ns}$. For every node, compute

$$AT(v) = \max_{u\to v}(AT(u) + d_{uv}),$$

then propagate required times backwards using

$$RAT(v) = \min_{v\to u}(RAT(u) - d_{vu}).$$

Report AT, RAT, and slack for every node.

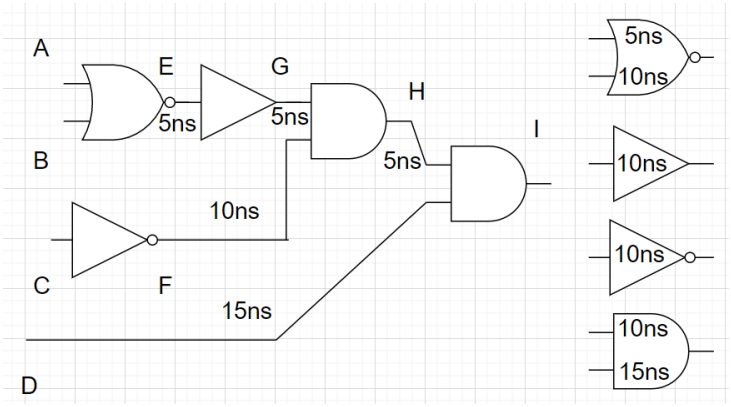


Figure 2: Combinational timing graph and gate delays

Problem 3: Setup analysis before CPPR

The clock period is $T = 20\,\text{ps}$. Every cell has early delay 5 ps and late delay 10 ps, net delay is zero, and setup time is 5 ps. The launch edge is the rising edge at time 0 and the capture edge is the next rising edge at time T. Use late data arrival and the specified early capture-clock arrival for setup analysis. Compute early and late AT for all nodes in Figure 3. For nodes D and E, compute late RAT and pre-CPPR setup slack. Before CPPR, common clock-path delay is counted independently on the launch and capture sides.

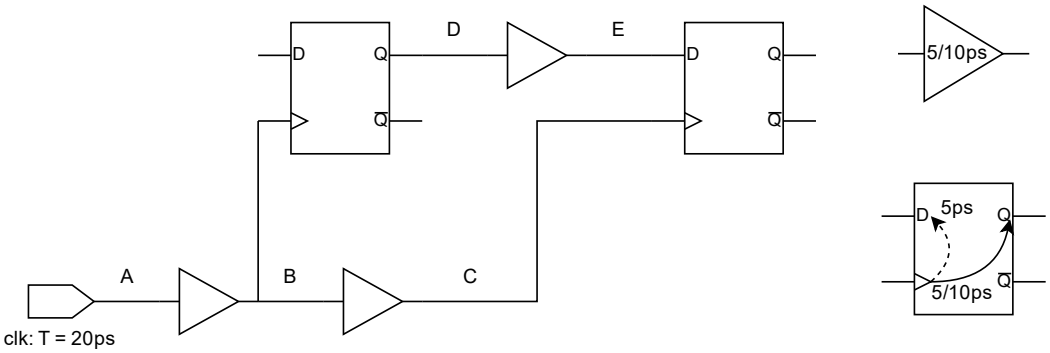


Figure 3: Clock and data paths

Problem 4: Inverted capture clock

Use the same delays and period as Problem 3. Replace the indicated capture-path buffer by an inverter. The inverter changes the active capture edge to the next falling edge, so the available capture time is $T/2$ plus the early capture-clock delay. Compute the new setup RAT at E:

$$RAT(E) = T/2 + AT_{early}(\textit{capture clock}) - t_{setup}.$$

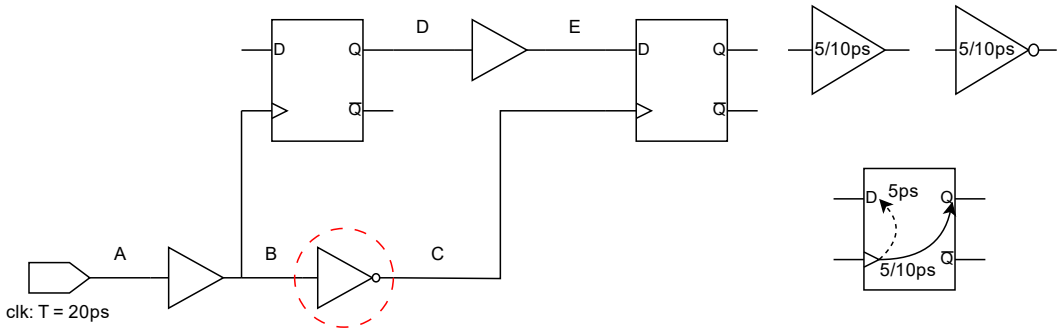


Figure 4: Capture path with inverter

Problem 5: Setup timing with CPPR

Complete `sta.py` to implement setup timing analysis with simplified CPPR.

Input. Use OpenROAD's existing post-CTS gcd case: `src/rsz/test/gcd_asap7_cts.v`, with its SDC and libraries under `test/`. The netlist already contains clock-tree buffers. The supplied `run_hw2.py` reads these files and supplies a data graph, a clock tree, late data delays, early/late clock delays, clk-q and setup times (ns). Run from the homework directory:

```
python3 run_hw2.py --openroad-root /path/to/OpenROAD
```

The default period is 0.28 ns and early/late cell-delay derates are 0.9/1.1. Use `-sta /path/to/sta` to select OpenSTA. Function interfaces are documented in `sta.py`.

Output. The runner writes `run_gcd/result.json` and `run_gcd/node_timing.json`. Report WNS/TNS and critical paths before and after CPPR, together with each launch–endpoint pair's slack, correction and common clock pin. The runner handles the file formats and saves node timing.

Requirements.

1. Implement timing propagation, CPPR and critical path tracing for one clock tree with noninverting buffers and rising-edge FFs. Analyze full-cycle setup timing only.
2. Compute the submitted timing with your own implementation. OpenSTA is provided for input reading and result comparison.
3. Run OpenSTA yourself with the same period and derates to compare CPPR credits. Report an endpoint that changes its worst launch after CPPR.

Hints. Use Problem 2's forward AT/backward RAT propagation and Problem 3's setup relation. CPPR adds $\sum(d_{late} - d_{early})$ over the shared prefix of the launch and capture clock paths to the pre-CPPR slack. Track each launch separately; the worst path can change after correction. PI/tie launches receive zero credit; count each endpoint once in TNS.

Submission

Submit a zip named `hw2-YOURSTUDENTID.zip` containing:

- `hw2-YOURSTUDENTID.pdf`: answers to Problems 1–4 and a short report on Problem 5, covering your implementation and the results requested above.
- `src/`: your implementation.
- `README.md`: exact commands to run your code.